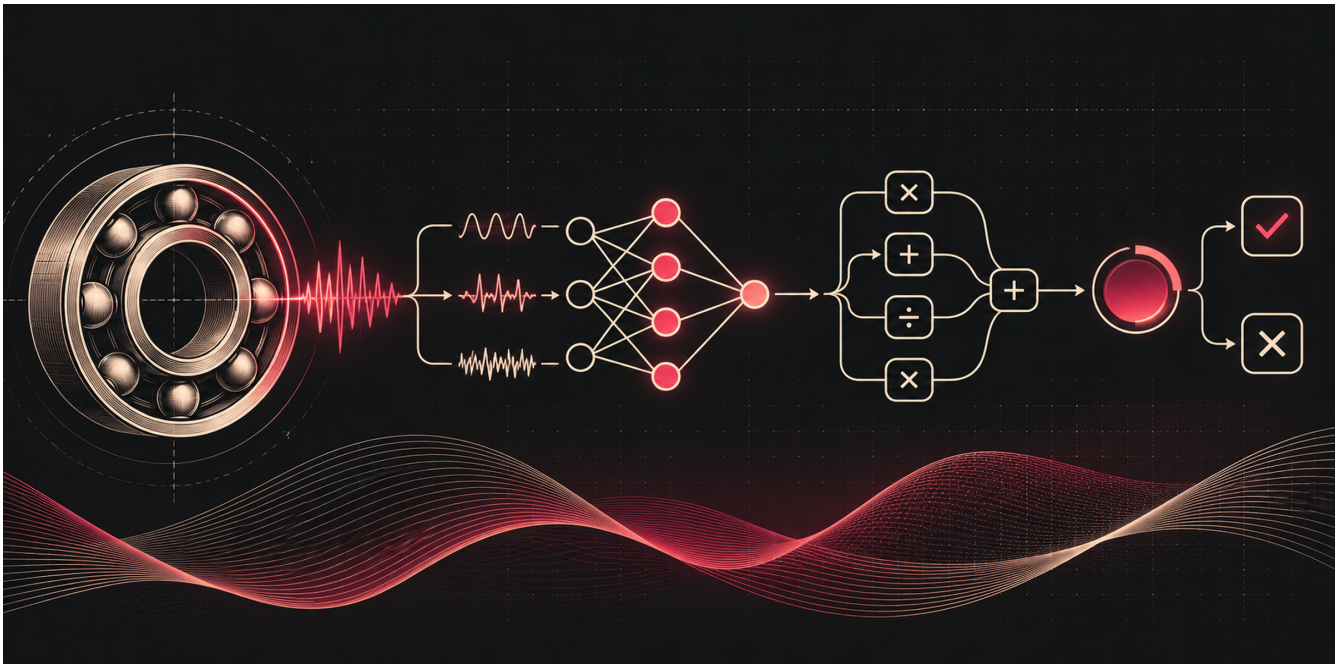
From Trained Neural Networks to Reusable Mathematical Execution Graphs

Industrial edge condition monitoring with original paceval SDK 4.25



Conceptual illustration: bearing features, neural inference, reusable mathematical execution and advisory decisions.

Research question

Can a small frozen trained neural network, its numerical feature normalization and its version-controlled advisory scoring be deployed as reusable paceval execution graphs while preserving mathematical results and allowing controlled model revisions?

Evidence classification

This report includes real execution with the user-provided Linux paceval shared library, a synthetic training dataset, two trained network revisions, real graph executions and independent numeric reference calculations. It does not establish industrial fault-detection validity or certified numerical error bounds.

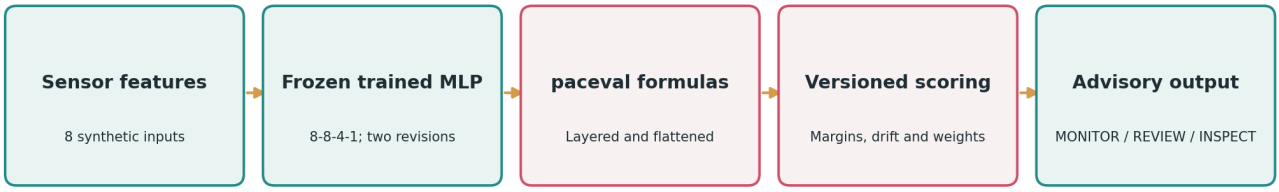
Research date: 13 November 2026 | Programming language: C++ | Data: entirely synthetic | Arithmetic: double precision

1. Executive summary

Industrial condition monitoring analyzes vibration, motor-current signals, temperatures and operating points to identify possible equipment deterioration. Compact trained neural networks are one candidate for classifying derived feature vectors. The special proposition of paceval is not network training: it is the deployment of frozen mathematical expressions as persistent, reusable computation objects alongside other application-specific formulas.

Executed item	Observed result
Persistent paceval graph handles	60
Two trained models, architecture	8 inputs → 8 → 4 → 1
Synthetic inputs / independent virtual bearing identities	1,536 / 48
Graph outputs and batch calls	117,897 / 148
Independent 80-digit high-precision comparisons	2,286
Largest audited network error	1.44e-15
Numerical outcome	PASS; no threshold flips in audited subset

Mathematical inference is one component of a governed monitoring workflow



The evaluation system separates external feature acquisition, frozen model weights, deterministic mathematical inference, rule evaluation and the final advisory policy.

The comprehensive trained-network experiment remains a controlled synthetic pilot. Industrial functional validation would require a permitted real dataset and tests on held-out physical bearings, including operating-condition shifts, feature validation and domain-specific error analysis.

2. Scientific foundation and contribution of the original paceval research

The accompanying paceval paper, Conversion of neural networks to closed-form expressions as human-readable mathematical functions (2015–2023), proposes recording scalar operations with a RecordFormula value and tracing a neural network dynamically through Julia/Flux. Each scalar stores its numerical value and the mathematical expression that produced it. The paper also describes splitting the output into formulas with intermediate variables at network-layer boundaries, avoiding repeated expansion of identical subexpressions.

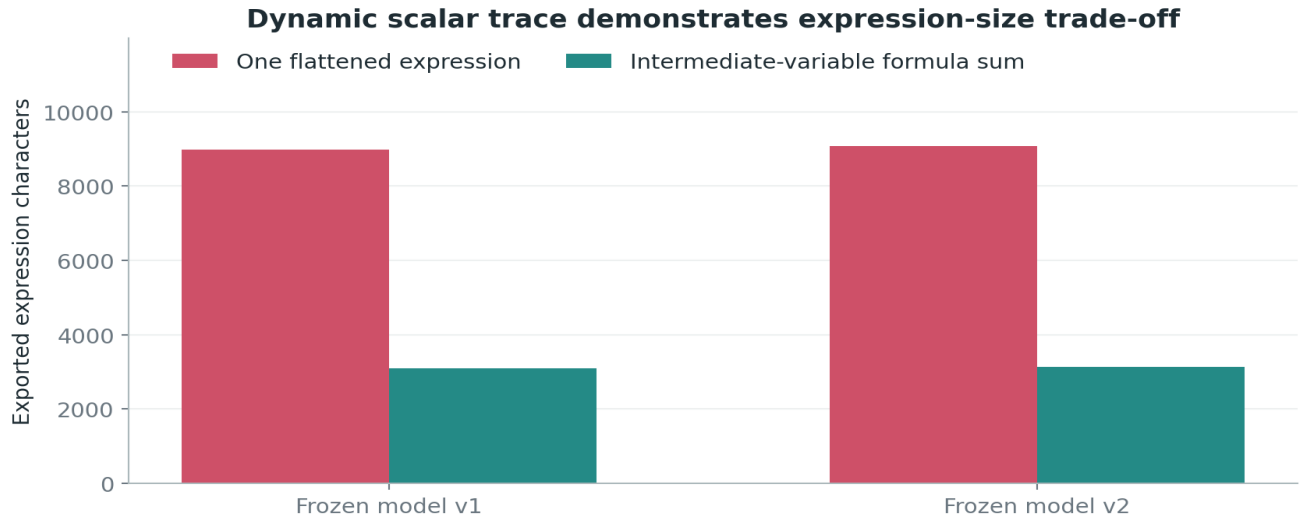
How this study extends the original method

• Replaces an untrained synthetic test network with two reproducibly trained, frozen 8–8–4–1 models. The models remain synthetic and have no established clinical or industrial diagnostic validity.

• Implements the same mathematical export idea through an explicit C++ architecture-aware exporter and a separately executable Python scalar operator tracer. No untested Julia, Flux or ONNX-to-paceval conversion is claimed.

• Executes normalizations, all neural layers, flattened output formulas and downstream advisory mathematics through the original paceval API. The host handles CSV, model lifecycle and discrete rule selection.

• Independently checks the complete computations and uses held-out synthetic bearing identities and shifted operating conditions to separate numerical parity from generalization quality.



Executed scalar-tracer output shows why explicit intermediate variables reduce duplicated textual formula length for this architecture.

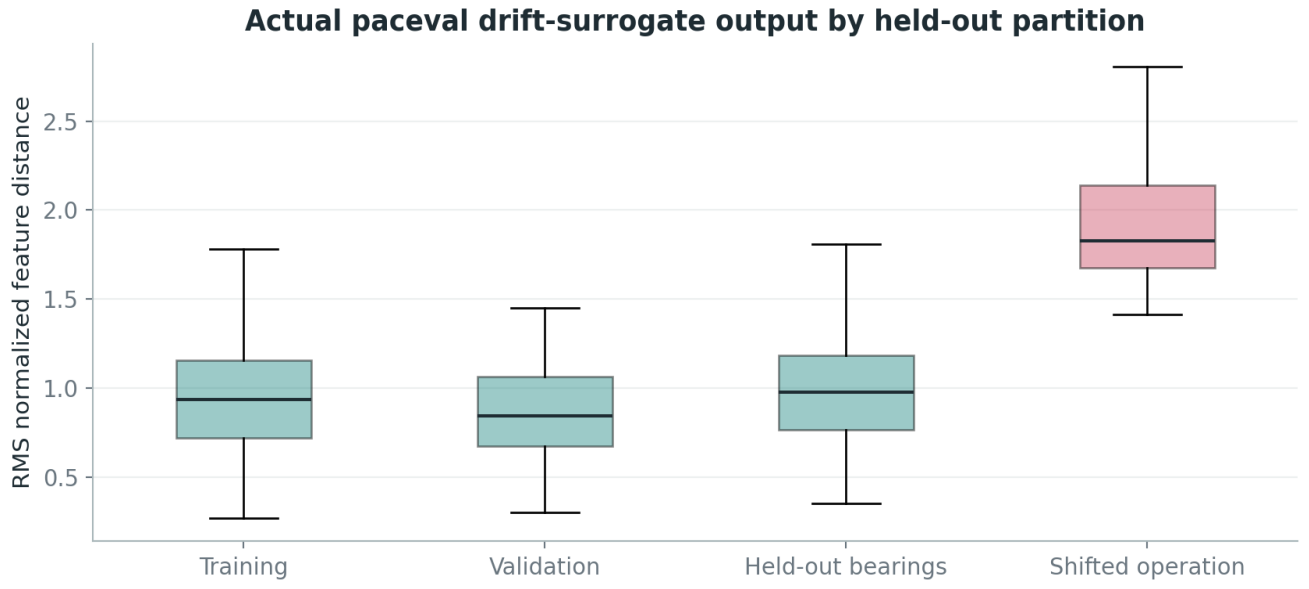
3. Industrial use case and dataset construction

A monitoring gateway might collect vibration acceleration, motor-current signals, rotational speed, load and temperatures. Conventional signal processing produces features such as RMS, crest factor, kurtosis and band energy. A compact frozen network transforms these features into an advisory score; a later rule layer controls whether operators are asked to inspect or review the measurement. The model does not actuate a machine.

Frozen synthetic experimental data

The generator creates 48 distinct virtual bearings, 32 windows per bearing. Their identities are split into 24 training groups, 8 validation groups, 8 ordinary held-out groups and 8 groups with deliberately shifted rotational operating conditions. Normalization mean and standard deviation are calculated only from the training groups. Each frozen revision is trained by deterministic batch Adam for 600 epochs using only the training group.

Eight input features are vibration RMS, crest factor, kurtosis, vibration band energy, motor-current RMS, harmonic-current ratio, bearing temperature and shaft speed. Labels are synthetic and follow the generator rules: they are not measured physical faults. Fault-prediction percentages in the following plots therefore characterize only this controlled simulation.



RMS feature-distance surrogate calculated through paceval. A larger shifted-group distance reflects the chosen synthetic operating-point construction, not a validated out-of-distribution detector.

4. Mathematical model and explicit execution ownership

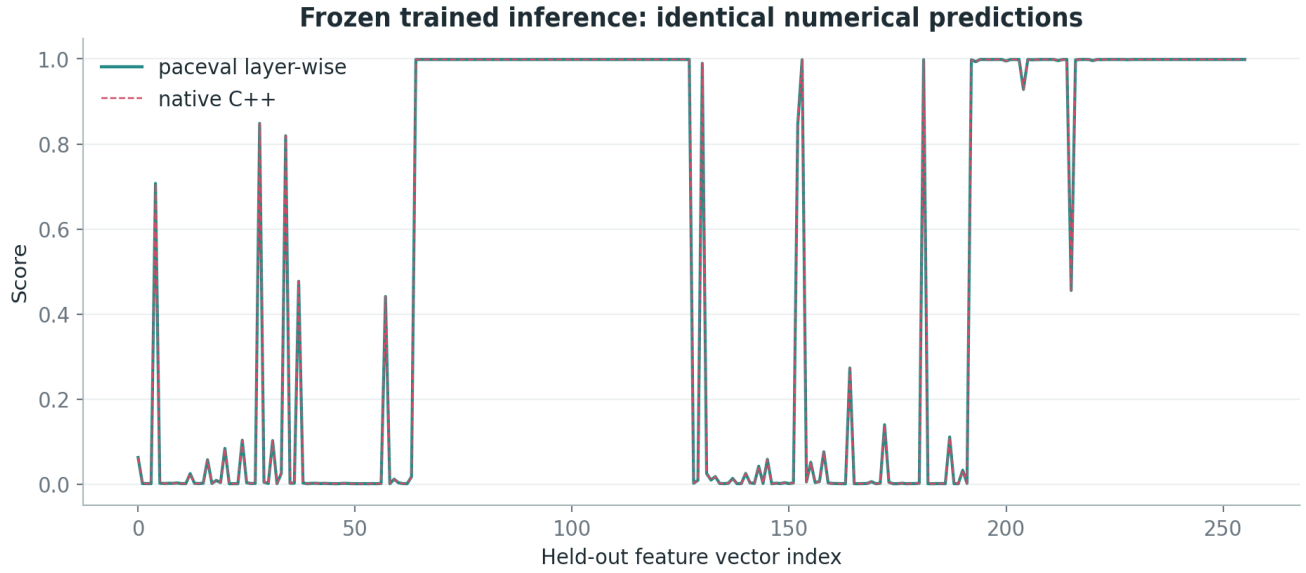
Frozen network

For feature vector f, the normalization is $x_i = (f_i - \mu_i)/\sigma_i$. The hidden layers are $h_1 = \text{sigmoid}(W_1 x + b_1)$ and $h_2 = \text{sigmoid}(W_2 h_1 + b_2)$. The network output is $s = \text{sigmoid}(w_3^T h_2 + b_3)$, where $\text{sigmoid}(z) = 1/(1 + \exp(-z))$. All weights and constants are exported with round-trip binary64 decimal precision.

Execution architecture

Eight reusable paceval graphs normalize raw features. Each trained model uses eight first-layer graphs, four second-layer graphs, one output graph and one alternatively flattened output expression. The flattened and layered paths represent the same frozen network using different graph organizations. Twenty-four additional graph handles, twelve per model version, implement decision margins, review bands, model-aware operating corrections, advisory scoring, feature distance and analytic derivatives.

The C++ host intentionally retains loading and validation, model/graph registry, chunk orchestration, outlier rejection and the final discrete MONITOR / REVIEW / ADVISE_INSPECTION action selection. Model training is offline in Python. No automatic differentiation or TINC-based certified interval result is claimed.

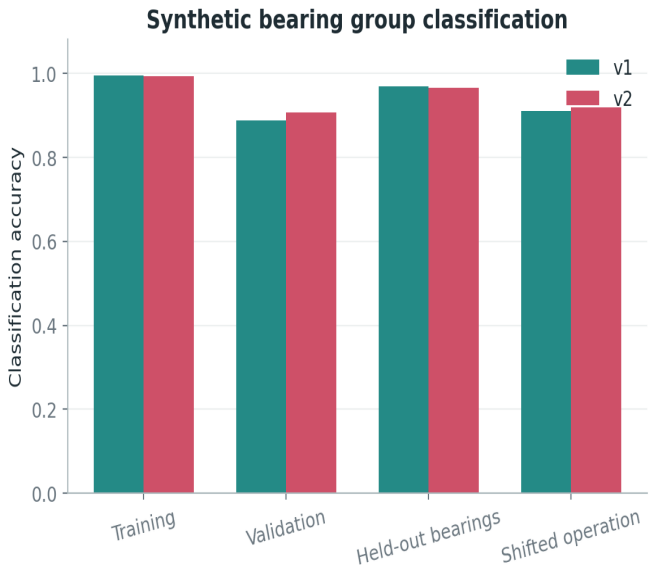


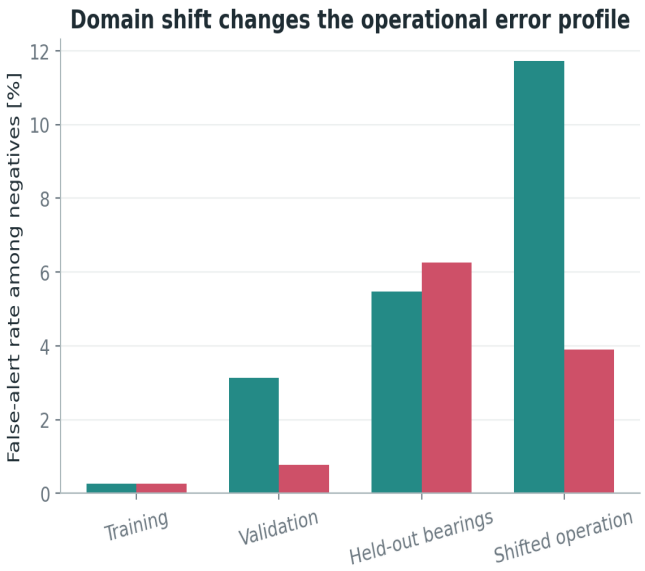
Actual original-SDK paceval outputs and an independently coded C++ forward pass overlay on held-out synthetic bearing vectors.

5. Synthetic trained-model results

Classification quality is a property of the chosen synthetic feature generator and trained weights, not an improvement introduced by paceval. Both the independently implemented native calculation and paceval evaluate the same frozen mathematics. The independent held-out and shifted partitions intentionally challenge the generator-specific learned relationship.

Model	Partition	Accuracy	False-alert rate
v1	Validation	88.67%	3.12%
v1	Held-out	96.88%	5.47%
v1	Shifted operation	91.02%	11.72%
v2	Validation	90.62%	0.78%
v2	Held-out	96.48%	6.25%
v2	Shifted operation	91.80%	3.91%



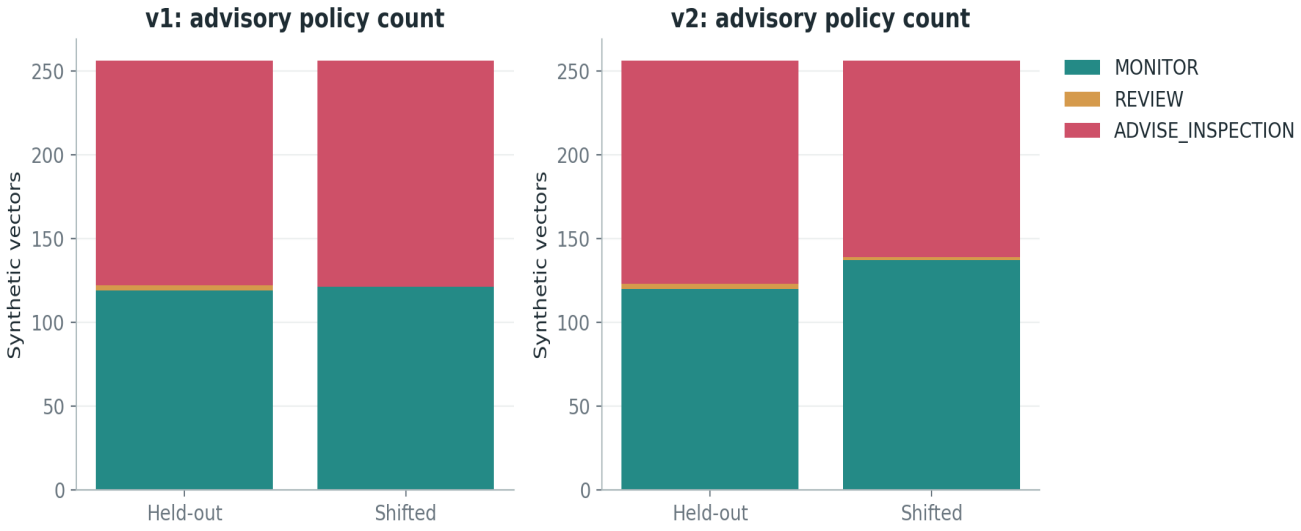


Observed synthetic classification performance differs across partitions. The original research question requires independent bearing-level holdout, not random window-level leakage.

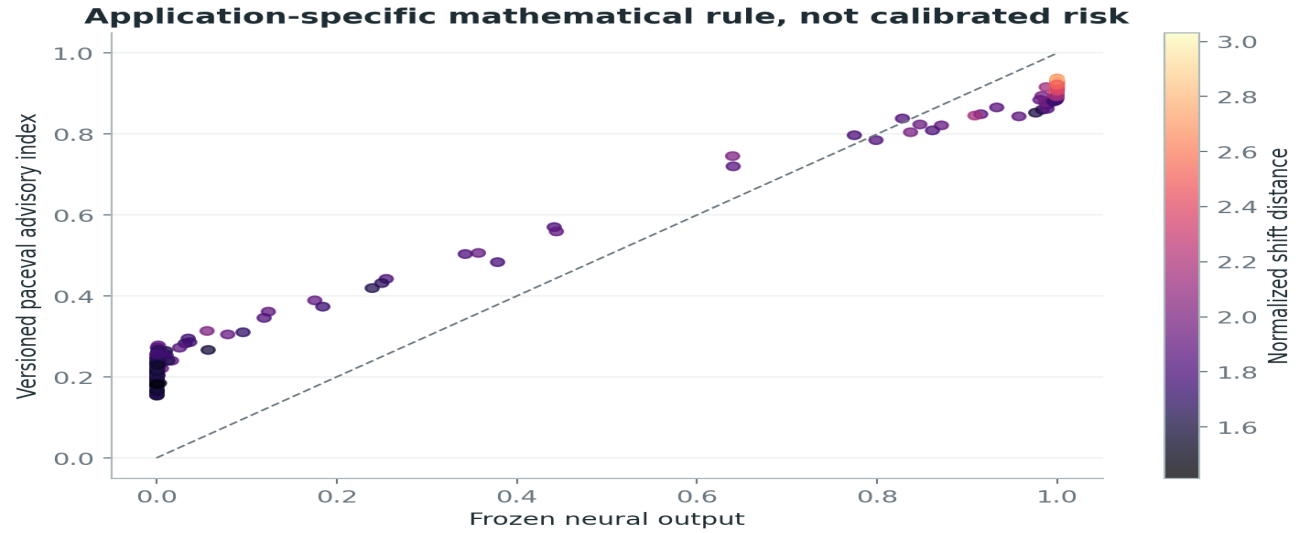
6. Decisions, domain shift and limitations

The operational postprocessing uses a version-controlled mathematical rule, which is an illustrative advisory score, not a calibrated probability of failure. For score s, normalized vibration x_0, shaft speed x_7 and temperature x_6, the prototype constructs $z = (s - 0.5) + a \cdot x_0 - b \cdot x_7 + 0.02 \cdot x_6$ and $q = \text{sigmoid}(3z)$. The coefficient pair (a,b) is (0.08,0.035) for v1 and (0.11,0.025) for v2. These coefficients are research fixtures rather than parameters fitted to actual maintenance costs.

The signed margin $m = s - 0.5$ and review width 0.08 define an advisory rule: REVIEW when |m| is below 0.08; otherwise ADVISE_INSPECTION for nonnegative m and MONITOR for negative m. No production action is automated, and an equipment safety controller remains external.



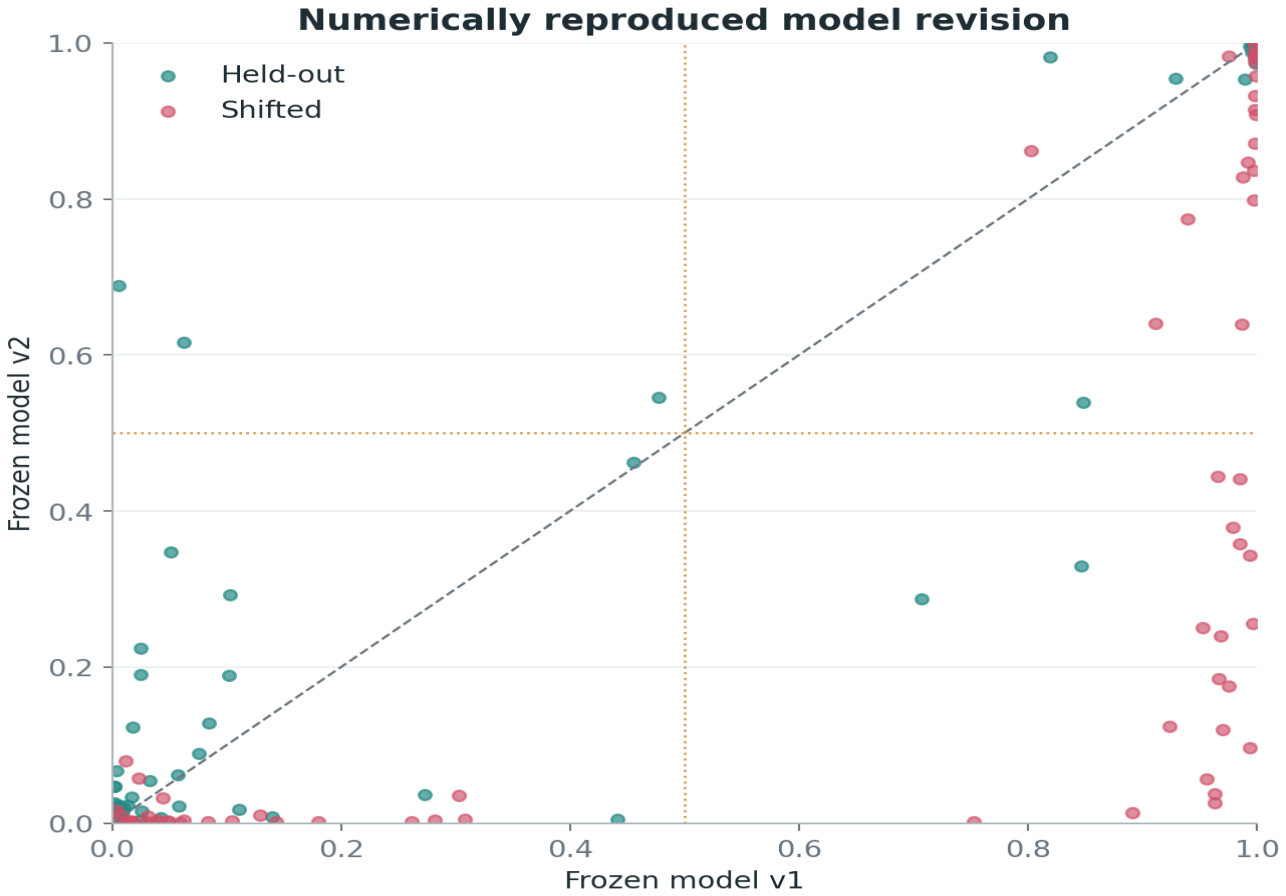
Actual paceval score margins with deterministic C++ category selection. Changes reflect synthetic features and model coefficients.



The operating-point correction changes the advisory index even with the same trained network output; it must be separately audited and version-controlled.

7. Versioned mathematical model updates

Two frozen models are simultaneously available rather than modifying the active graph in place. The project emits a machine-readable registry of every expression, a model manifest with data and weight SHA-256 hashes, a complete score-difference file and an independent dynamic-trace audit. This demonstrates predictable deployment artifacts and traceability, not a quantified reduction in real engineering release costs.

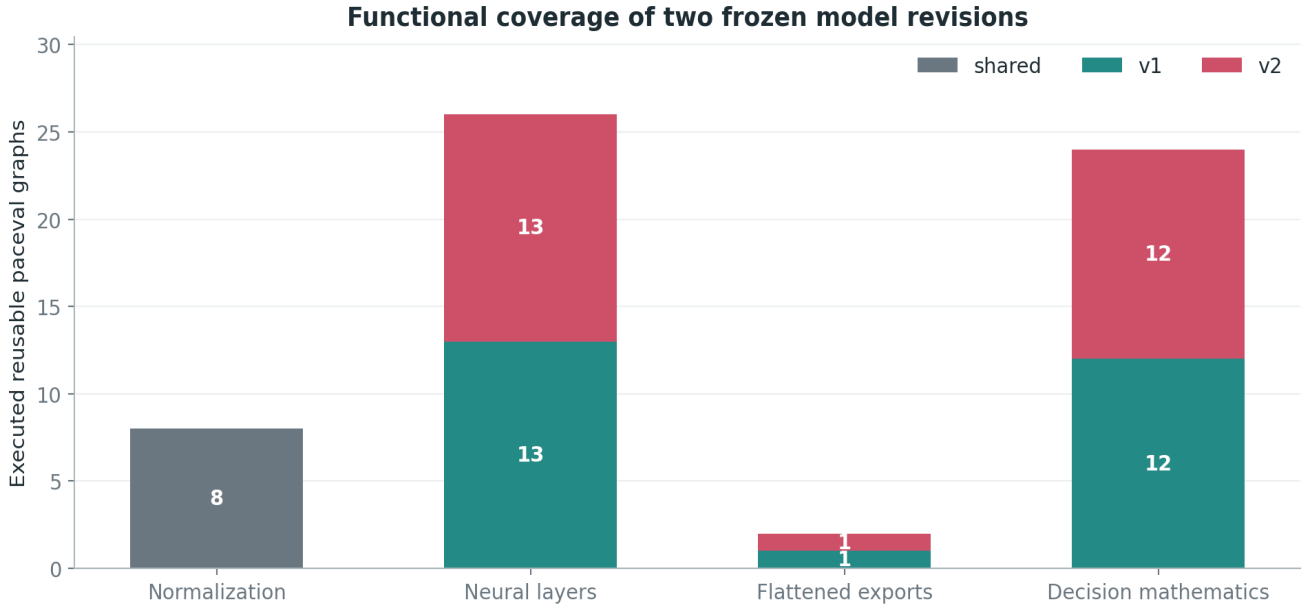


Recomputed frozen-network revision across held-out and shifted synthetic cases. Differences are expected because weights were independently trained. The diagonal indicates identical outputs.

Any production deployment would still need a versioned data schema, signed binaries or formula assets, update/rollback procedure, operational monitoring, and separation between training, numerical evaluation and safety logic.

8. Executed paceval functionality and model portability

The central experimental result is the successful representation and execution of an entire frozen, trained neural model together with preprocessing and mathematical decision rules using the original paceval API. The model is not retrained or recompiled when new feature vectors are evaluated. Both trained revisions retain independent graph registries and can be selected without modifying the C++ orchestration algorithm.



Functional inventory of all 60 executed persistent paceval computation objects. These are actual graphs registered through the user-supplied original SDK, not abstract diagrams.

Executed mathematical component	Verified implementation
Training-group feature normalization	8 shared persistent paceval graphs
Frozen two-layer MLP	13 layer-structured graphs per model revision
Alternative closed output form	1 flattened graph per revision
Margins, correction, scores and derivatives	12 advisory graphs per revision
Input validation and discrete policy	C++ host, independently traceable
Two model revisions	Available concurrently; reproducible weight manifest

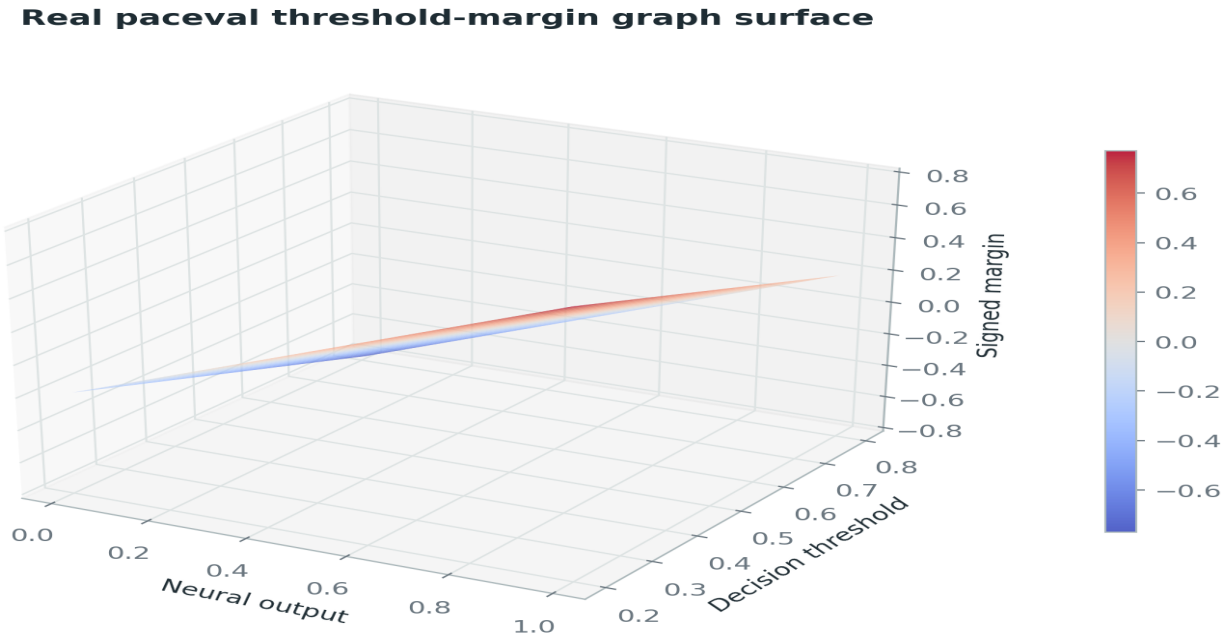
The scalar path and the multi-graph batch path are both exercised in the C++ study. Mathematical equivalence is checked with independently implemented calculations. An FPGA/ASIC execution-graph implementation is a separate architectural research direction and was not evaluated in this feasibility experiment.

9. Numerical verification and decision stability

Every synthetic row and both frozen versions were compared against independently coded native C++ inference inside the application. The layered output was also compared to the flattened mathematical export. A separate Python program independently implemented 80-digit decimal inference, training-fold normalization, versioned advisory index and inspection-priority formulas.

Independent check	Outcome
Numerical precision	80 decimal digits
Number of selected samples per frozen model	381
High-precision checks, including composed outcomes	2,286
Maximum network absolute difference	1.441e-15
Threshold flips among audited samples	0
C++ native vs paceval and flattened path	Every input, both versions: PASS

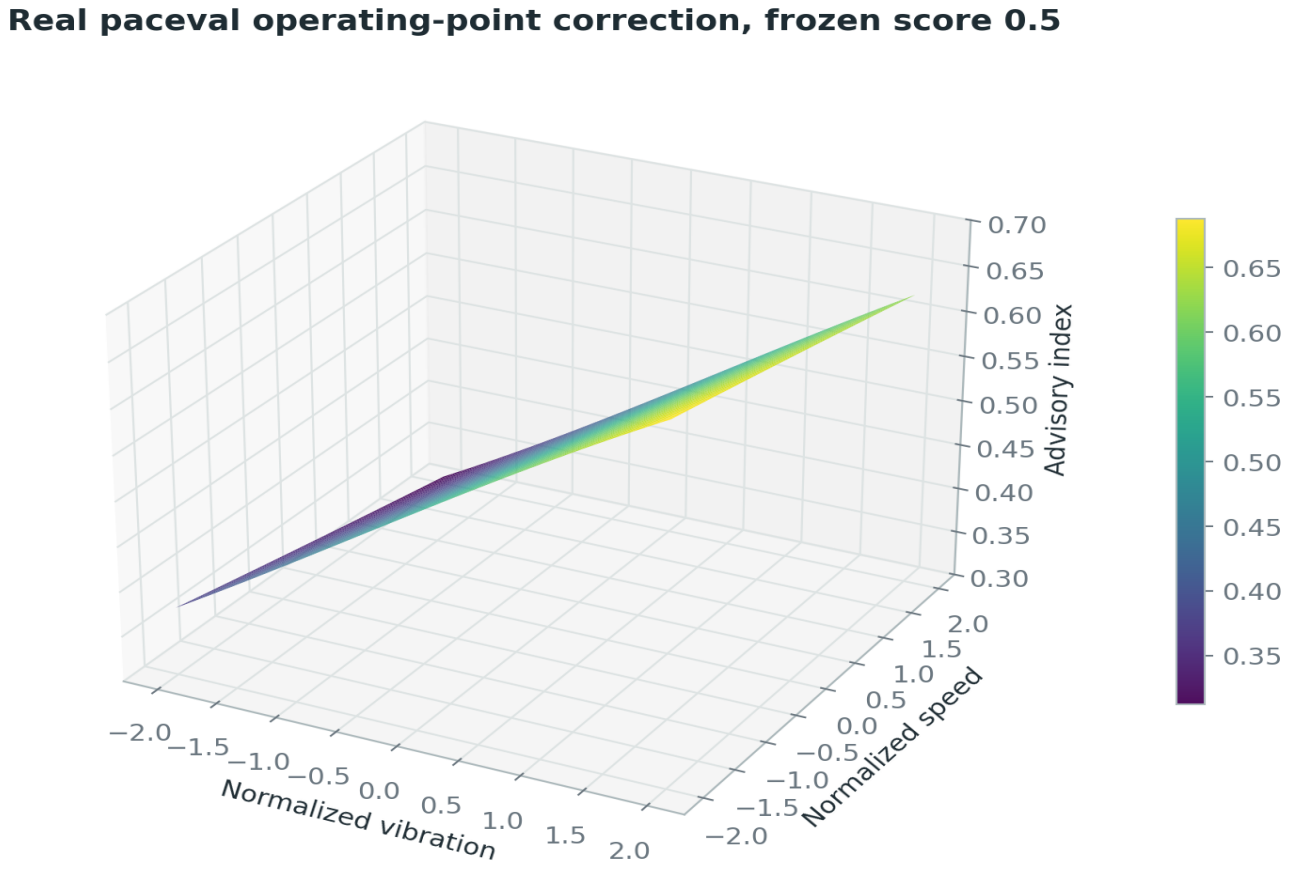
These checks are finite-sample empirical audits. They do not prove a universal numerical error bound, certified threshold stability, clinical or industrial predictive validity, or TINC interval certification. Since the score distribution is sharply concentrated near 0 and 1, specific threshold stress tests should accompany any future real deployment.



Executed paceval margin function over score and threshold values. The planar surface is mathematically expected because m = score − threshold.

10. Mathematical integration and research extensions

A functional research extension is to deploy the neural output together with operating-point corrections, nonlinear advisory scoring, constraint margins and other plant-specific mathematical functions in one versioned paceval execution layer. The present prototype demonstrates these classes of formulas in a small reproducible synthetic workflow.



Nonlinear operating-point advisory surface: executed original paceval graph at fixed network score, varying normalized vibration and speed. Formula revisions can change the surface without changing the model architecture.

Falsifiable next experiment

Train a compact network on permitted industrial bearing measurements and keep physical bearing identities disjoint across training and independent testing. Validate the automatic export against the originating model at identical input windows, operators, weights and normalization. Test revision and rollback semantics, input rejection, decision-margin stability and operating-point shifts. An FPGA/ASIC implementation of the same execution graphs is a plausible future architecture but has not been demonstrated in this experiment.

11. Reproduction, boundaries and scientific sources

Reproduction instructions

The complete ZIP contains src/edge_neural_formula.cpp, original SDK C++ API glue files, the source-only synthetic trainer, frozen decimal weights, numerical parity outputs, the RecordFormula-style operator tracer, high-precision reference code, the figure generator and this report. The proprietary/runtime binary is deliberately excluded from the distributable ZIP and must be provided separately.

1. Build with cmake -S src -B build -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so, then cmake --build build -j.

2. Execute ./build/edge_neural_formula . 128 from the project root. Run the independent scripts python3 src/verify_independent.py and python3 src/trace_neural_formulas.py.

3. Regenerate plots with python3 src/make_figures.py. Reproducing the frozen training data requires Python NumPy; executing C++ itself requires only the original paceval SDK, libstdc++ and the Linux runtime.

Scientific and operational boundaries

No Paderborn bearing records, real patient information, operating plant measurements or third-party proprietary models are shipped. The model is not a validated failure detector. The historical paceval paper illustrates Julia dynamic tracking; this research uses a compatible but independent miniature Python tracer and a C++ graph generator. A flattened expression is mathematically equivalent for this specific supported MLP, not for arbitrary convolutional or dynamic neural architectures.

Selected references

[1] paceval. Conversion of neural networks to closed-form expressions as human-readable mathematical functions. Original user-supplied research paper (2015–2023), pp. 6–11. https://github.com/paceval/paceval/tree/main/paceval%20and%20artificial%20intelligence

[2] Lessmeier et al. Paderborn University Bearing DataCenter. Vibration and motor-current research data, CC BY-NC 4.0. https://mb.uni-paderborn.de/en/kat/research/bearing-datacenter

[3] Goodarzi, Schütze and Schneider (2025). Domain shifts in industrial condition monitoring: a comparative analysis of automated machine learning models. J. Sens. Sens. Syst. 14, 119–132. https://doi.org/10.5194/jsss-14-119-2025

[4] paceval C/C++ API and computation-handle documentation. https://paceval.com/api/

[5] Earlier paceval research: From Neural Networks to Reusable Mathematical Execution Graphs, user-supplied research report, 13 November 2026.

12. Interpretation and conclusions

The integrated research demonstrator verifies a technically important proposition: an already trained compact sigmoid network and its surrounding mathematical decision functions can be represented as persistent, reusable mathematical graphs executed by the original paceval engine. Training artifacts are separate from inference; each model revision has stable numerical weights, and every mathematical component is recorded in an execution inventory.

What was demonstrated

• Working original paceval C++ API, reusable expression handles, multi-calculation and multi-graph evaluation, two frozen trained network revisions, neural layer boundaries and flattened export.

• Reproducible normalization, output scoring, margins, shifted-feature descriptors and analytical score sensitivities as actual paceval graphs.

• Independent parity against native C++, a RecordFormula-inspired operator trace and high-precision reference computations.

What is not yet demonstrated

• A real rolling-bearing failure classifier validated on an industrial test set, calibrated risk probability, or approved safety function.

• A field-validated predictive maintenance system using recorded data from physical bearings under changing operational conditions.

• A general Julia/Flux or ONNX-to-paceval converter, formally certified numerical bounds, or a completed FPGA/ASIC execution prototype.

Scientific conclusion: The original formula-export proposal is strengthened by a controlled trained-model example and independently validated inference. The most promising follow-up is a verified conversion pipeline for permitted real industrial data, followed by rigorous functional and numerical tests across model revisions and operating conditions.